POCKETSMART AI

YOUR SMART BUDGET & RECOMMENDATION ASSISTANT

AI-powered • Budget-aware • Personalized recommendations

TEAM DETAILS

TEAM ID	SWTID-2026-1446
TEAM LEADER	Bairojabanu K
TEAM MEMBER	Gobika J
TEAM MEMBER	Kanimozhi K
TEAM MEMBER	Kaviya V

INTRODUCTION

PocketSmart AI is a smart, AI-powered budget and recommendation assistant designed to help users plan and manage their spending across different lifestyle needs.

The system provides personalized, budget-aware recommendations for home interior planning, party planning, and jewelry selection.

It uses user preferences, budgets, contextual details, and optional images to generate suitable suggestions.

By combining FastAPI with Gemini 1.5 Flash Pro and integrating information from platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO, PocketSmart AI aims to make planning easier, faster, and more convenient while helping users stay within their defined budgets.

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description

Managing budgets across different life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. **PocketSmart AI** addresses this challenge through a **GenAI-powered, cross-platform recommendation system** that delivers personalized, budget-based suggestions for products and services.Using **Gemini 1.5 Flash Pro**, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as **Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO**, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets.PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine that bridges multiple categories and e-commerce ecosystems.

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the **Home Budget Planner**. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quantities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon.Recommendations are balanced across functionality, style, and price, ensuring users can decorate their spaces beautifully without overspending.

Scenario 2: AI-Based Party Budget Planning

In the **Party Planner**, users input their total budget, guest count, event type, and venue details. Based on these inputs, the AI allocates the budget proportionally across **catering, decoration, and entertainment**, sourcing options from vendors listed on Swiggy, Zomato, and even accommodation services like OYO.The system tailors suggestions based on event type (e.g., birthday, corporate, wedding), helping users organize a successful event without financial guesswork.

Scenario 3: Jewelry Recommendations for Occasions

The **Jewelry Budget Planner** allows users to enter a budget, select the occasion, and define style preferences. Users can optionally upload an outfit image for the AI to consider color coordination and aesthetics. Gemini 1.5 Flash Pro analyzes the data and offers jewelry options from platforms like Amazon and Flipkart, tailored to occasion and style.This ensures that users receive elegant, matching jewelry suggestions that fit both their personal style and financial plan.

Architecture Overview

PocketSmart AI is an intelligent, modular platform designed to generate personalized, budget-aware recommendations across domains like home interiors, party planning, and jewelry selection. It integrates a lightweight Flask backend with Google's **Gemini 1.5 Flash Pro**, a powerful multimodal AI model that processes user inputs (text, numbers, and images) to deliver smart recommendations.

Core Technologies

- **Flask**: Backend service layer for request handling, routing, and session control.
- **Gemini 1.5 Flash Pro**: AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
- **Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.)**: Product and service data sourcing.
- **Frontend (HTML/CSS + JS)**: Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component	Description
Frontend UI	Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category.
Flask Backend	Manages user sessions, routes (`/generate-home`, `/generate-party`, `/generate-jewelry`), and handles communication between UI and Gemini AI.
Gemini AI Layer	Processes inputs using Gemini 1.5 Flash Pro to:

PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...



Pre-requisites

1. IBM Cloud Account Setup: IBM Cloud Console
2. Watsonx Access and Configuration: Watsonx Overview
3. Python Basics: Python.org
4. FastAPI Framework: FastAPI Docs

Project Workflow

1. Gemini AI Setup & Initialization

- **Activity 1.1**: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- **Activity 1.2**: Configure API keys, usage limits, and test integration for multimodal support (text + image).
- **Activity 1.3**: Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development

- **Activity 2.1**: Design Flask-based backend structure for routing, session control, and planner modules.
- **Activity 2.2**: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.
- **Activity 2.3**: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)

- **Activity 3.1**: Define Flask routes like `/generate-home`, `/generate-party`, `/generate-jewelry`.
- **Activity 3.2**: Organize backend using modular structure:
  - `routes/`: Endpoints
  - `services/`: Recommendation logic & AI calls
  - `models/`: Input/output schemas
- **Activity 3.3**: Setup CORS headers for frontend communication and implement basic session management.
- **Activity 3.4**: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

- **Activity 4.1**: Build responsive HTML forms for:
  - Home Interior Planner (room details, quantities)
  - Party Planner (event type, budget, guests)
  - Jewelry Planner (budget, occasion, outfit image upload)
- **Activity 4.2**: Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization

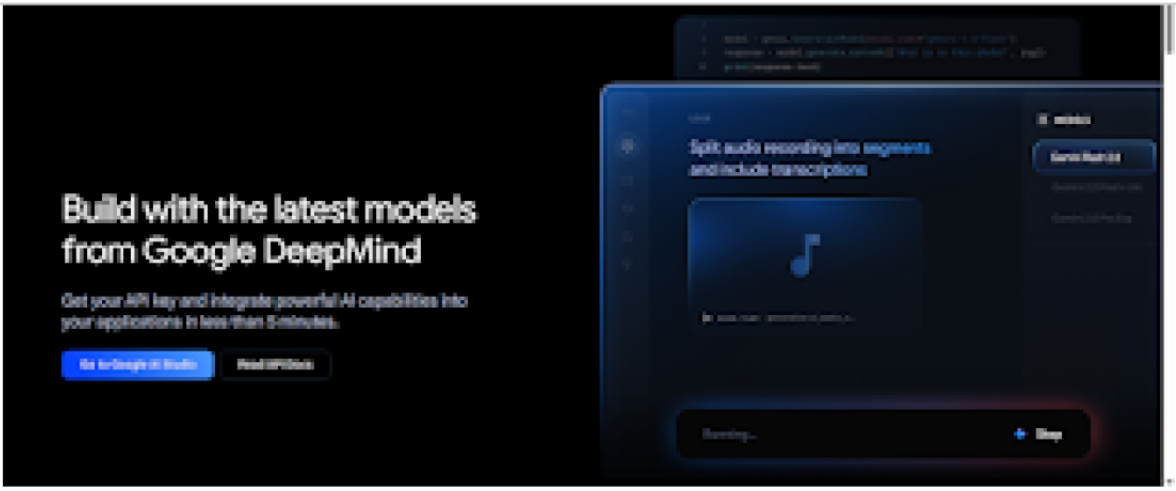
- **Activity 5.1**: Perform testing with real-world use cases across all three planners.
- **Activity 5.2**: Evaluate Gemini response quality, budget adherence, and platform accuracy.
- **Activity 5.3**: Optimize prompts and add validations to prevent input edge-case failures.
- **Activity 5.4**: Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization

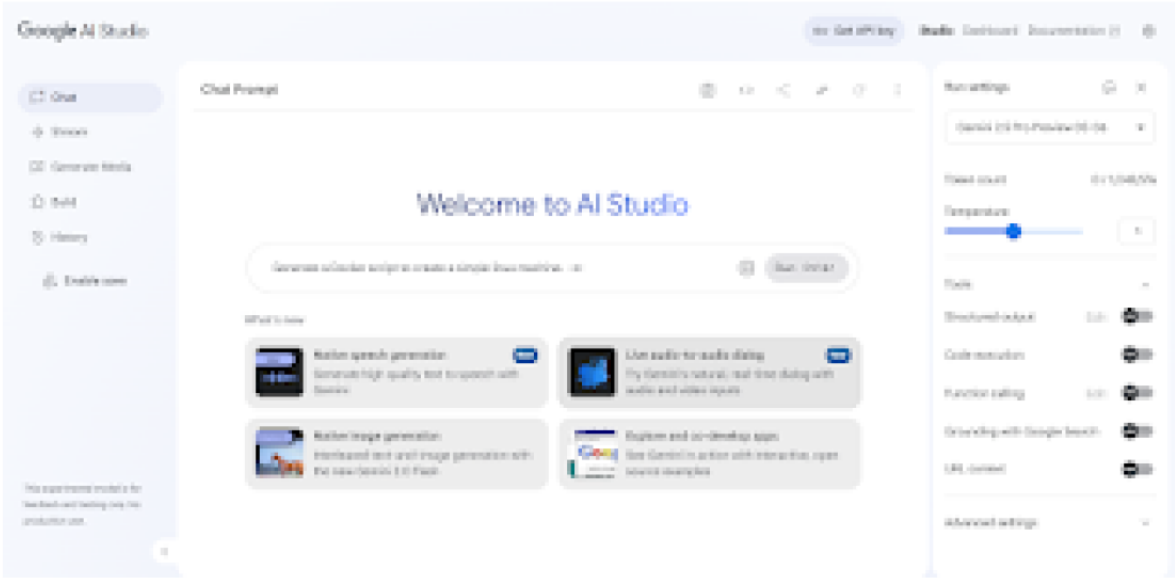
In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **PocketSmart AI**. The platform relies on **Gemini 1.5 Flash Pro**—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner).Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

- **Activity 1.1: Set up Google Cloud account and enable Gemini API**

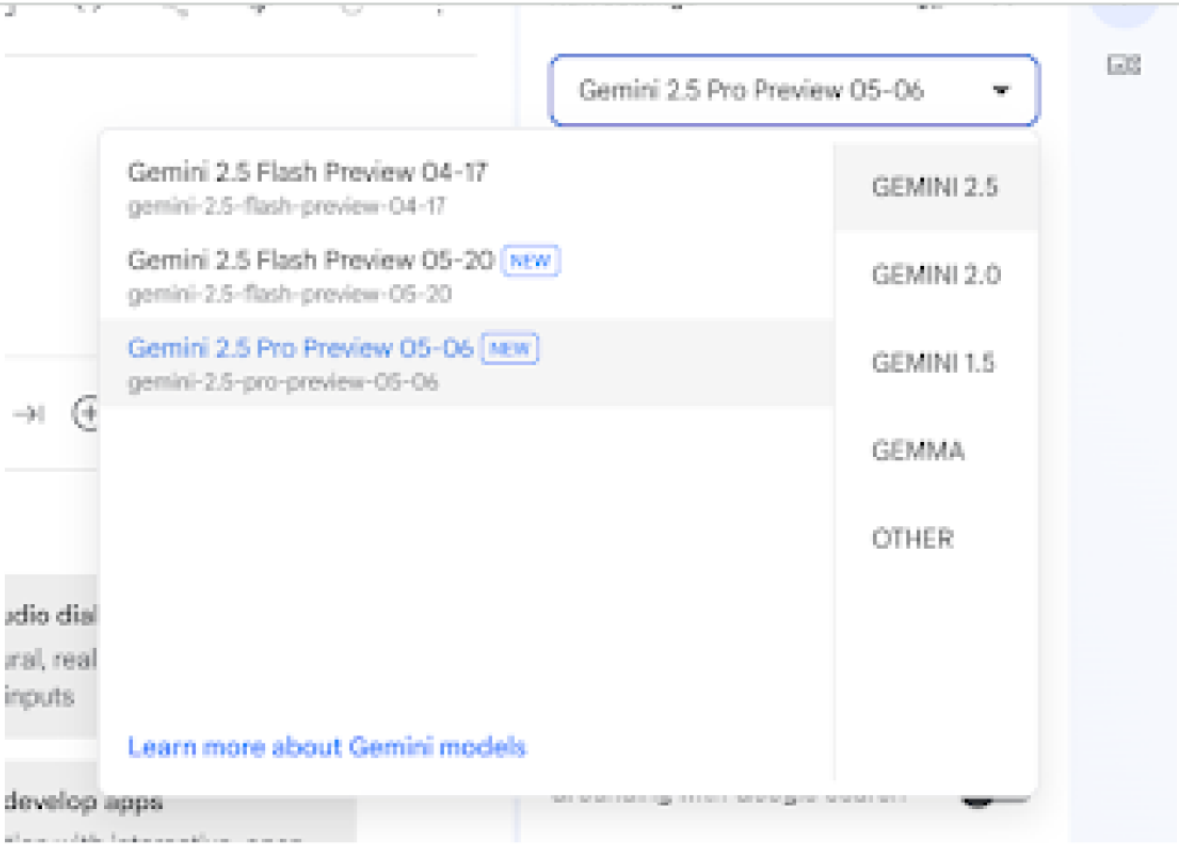
  - Sign in or create a Google Cloud account via console.cloud.google.com.



- Accept all terms and conditions and click on Get API key



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

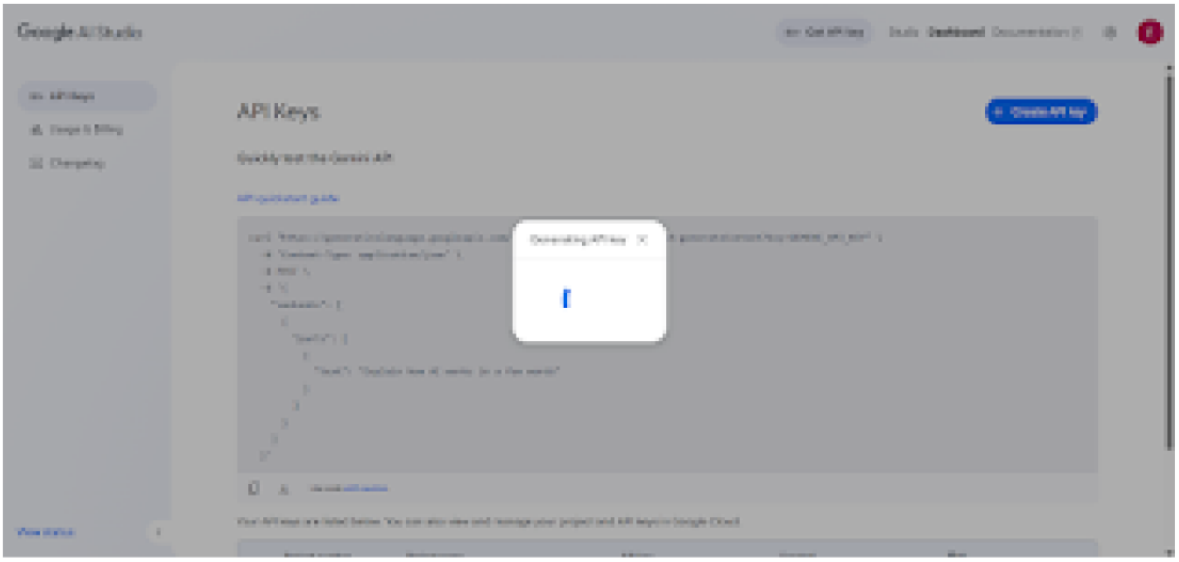


Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key



- Once click on API key you will the api key and make sure to copy the API key



PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...

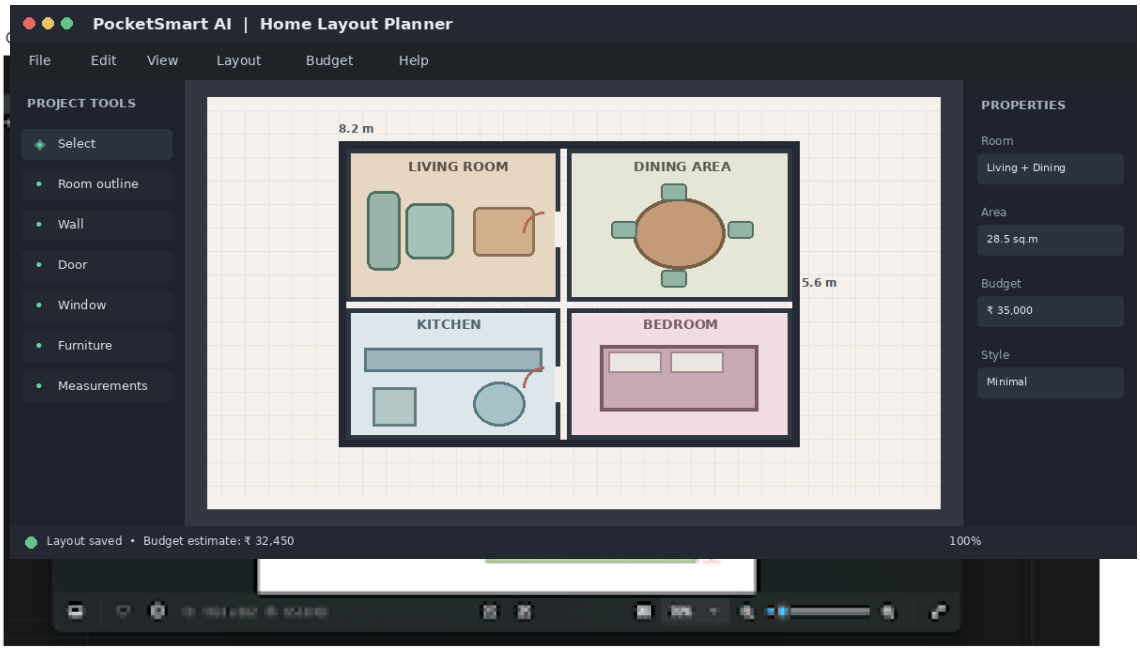
```python
from fastapi import APIRouter, UploadFile, File, Form, HTTPException
from PIL import Image
from io import BytesIO

router = APIRouter()

@router.post("/multimodal-recommendation")
async def multimodal_recommendation(
    prompt: str = Form(...),
    image: UploadFile = File(...)
):
    # Accept only supported image formats
    if image.content_type not in ("image/png", "image/jpeg"):
        raise HTTPException(status_code=415, detail="Unsupported image")

    image_bytes = await image.read()
    try:
        Image.open(BytesIO(image_bytes)).verify()
    except Exception:
        raise HTTPException(status_code=400, detail="Invalid image")

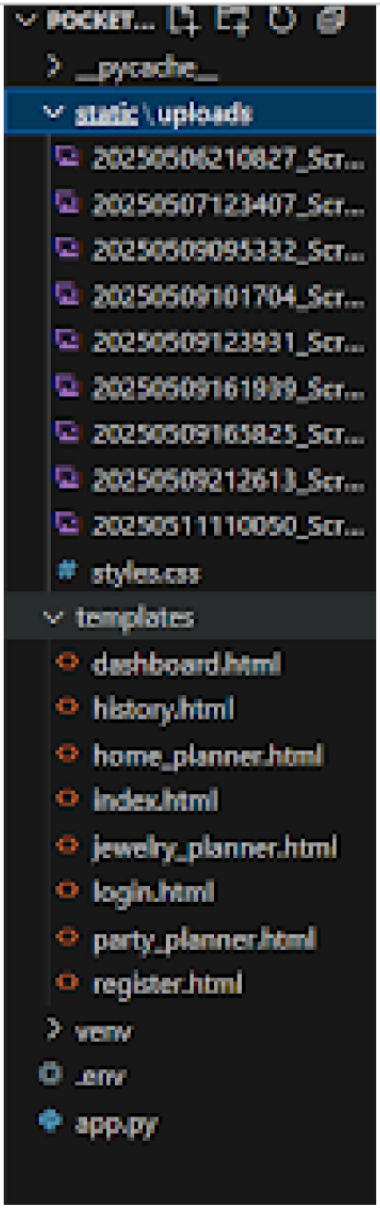
    result = await generate_multimodal_response(prompt, image_bytes)
    return {"prompt": prompt, "recommendation": result}
```



MILESTONE 2: Core Functionalities Development

This milestone focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations.The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (gemini_utils.py) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure



Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

- PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...

```
import os
from fastapi import FastAPI, Request, Depends
from fastapi.templating import Jinja2Templates
from fastapi.middleware.cors import CORSMiddleware
from dotenv import load_dotenv
from pathlib import Path
from gemini_utils import get_home_recommendations
```

· Fast API Initialization

```
# Initialize PocketSmart AI application
load_dotenv()

app = FastAPI(title="PocketSmart AI Budget Planner")

BASE_DIR = Path(__file__).resolve().parent
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
# Load environment settings
SECRET_KEY = os.getenv("SECRET_KEY", "change-this-key")
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")

if not GEMINI_API_KEY:
    raise RuntimeError("GEMINI_API_KEY is not configured")
```

Activity 2.2: Main Functionality code routings

· /generate-home: Generates home interior recommendations (furniture, decor, lighting)

```
@app.post("/generate-home")
async def generate_home(request: Request, current_user=Depends(get_current_active_user)):
    data = await request.json()
    budget = data.get("budget", 0)
    preferences = data.get("preferences", {})

    recommendations = get_home_recommendations(
        budget=budget, preferences=preferences
    )
    return {"status": "success",
            "recommendations": recommendations}
```

· get_home_recommendations() is used to generate budget-based suggestions

• linking product websites like Amazon, Flipkart, etc. to get product suggestions

```
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

class BudgetInput(BaseModel):
    total_budget: float
    category: str = "general"
    preferences: str = ""

@router.post("/recommend-products")
async def recommend_products(data: BudgetInput):
    prompt = (
        f"Suggest products available in India under Rs.{data.total_budget}. "
        f"Category: {data.category}. Preferences: {data.preferences}. "
        "Return product name, price, store and product URL."
    )
    result = await generate_ai_response(prompt)
    return {"recommendations": result}

def product_link(store: str, url: str):
    return {"store": store, "url": url}
```

```
def get_store_links(items):
    links = []
    for item in items:
        if item.get("url"):
            links.append({
                "name": item.get("name", "Product"),
                "price": item.get("price", 0),
                "store": item.get("store", ""),
                "url": item["url"]
            })
    return links
```

• /generate-party: suggests catering, venue, and decoration options for events

• get home recommendations by using the function

```
def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD to INR using the supplied exchange rate."""
    return amount_usd * exchange_rate

@app.get("/convert-currency")
async def convert_currency(amount_usd: float):
    return {"usd": amount_usd, "inr": usd_to_inr(amount_usd)}
```

```
@app.post("/party-planner")
async def party_planner(request: Request):
    data = await request.json()
    recommendations = await get_party_recommendations(data)
    return {"party_recommendations": recommendations}
```

```
async def get_party_recommendations(data):
    prompt = f"Suggest party recommendations for India with a total budget of {data['budget']}."
    prompt += f" Venue: {data.get('venue_type', 'any')}."
    prompt += f" Guests: {data.get('guests', 0)}."
    prompt += f" Food: {data.get('food', 'any')}."
    prompt += " Return venue, catering and decoration options with prices."
    result = await generate_ai_response(prompt)
    return result
```

PocketSmart AI – Budget Recommendation Coding

```
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

class ProductRequest(BaseModel):
total_budget: float
category: str = "general"

PRODUCTS = [
{"name": "Wireless Earbuds", "price": 1499,
"category": "electronics",
"store": "Amazon",
"url": "https://www.amazon.in/"},
{"name": "Smart Bottle", "price": 999,
"category": "general",
"store": "Flipkart",
"url": "https://www.flipkart.com/"},
{"name": "Power Bank", "price": 1799,
"category": "electronics",
"store": "Amazon",
"url": "https://www.amazon.in/"},
{"name": "Smart Watch", "price": 4999,
"category": "electronics",
"store": "Flipkart",
"url": "https://www.flipkart.com/"}
]

@router.post("/budget-recommendations")
async def budget_recommendations(data: ProductRequest):
results = []

for item in PRODUCTS:
category_match = (
data.category == "general"
or item["category"] == data.category
)

if item["price"] <= data.total_budget and category_match:
results.append({
"product": item["name"],
"price": item["price"],
"store": item["store"],
"url": item["url"]
})

results.sort(key=lambda x: x["price"])

return {
"budget": data.total_budget,
"category": data.category,
"recommendations": results
}
```

• Filters products according to the user's budget and category, then returns product name, price, store and shopping URL.

```
# Categorize the user's budget situation
def categorize_budget(total_budget: float) -> str:
    if total_budget < 5000:
        return "low"
    elif total_budget < 20000:
        return "medium"
    else:
        return "high"

# Select suitable products for the category
def get_budget_products(category: str):
    products = {
        "low": [
            {"name": "Wireless Earbuds", "max_price": 1500},
            {"name": "Smart Bottle", "max_price": 1000},
            {"name": "Power Bank", "max_price": 1800},
        ],
        "medium": [
            {"name": "Smart Watch", "max_price": 5000},
            {"name": "Bluetooth Speaker", "max_price": 4500},
            {"name": "Fitness Band", "max_price": 3000},
        ],
        "high": [
            {"name": "Premium Smart Watch", "max_price": 15000},
            {"name": "Laptop", "max_price": 50000},
        ],
    }
    return products.get(category, [])
```

- **using if condition to categorize the situation based products**

```
@router.post("/budget-products")
async def budget_products(data: BudgetInput):
    category = categorize_budget(data.total_budget)
    products = get_budget_products(category)

    # Keep only products within the user's budget
    results = [
        item for item in products
        if item["max_price"] <= data.total_budget
    ]

    return {
        "budget": data.total_budget,
        "category": category,
        "recommendations": results
    }
```

```
@router.post("/jewelry-recommendations")
async def jewelry_recommendations(data: JewelryInput):
    prompt = (
        f"Suggest jewelry for a budget of Rs.{data.total_budget}. "
        f"Style: {data.style}. Occasion: {data.occasion}. "
        "Return item name, material, estimated price and reason."
    )
    result = await generate_ai_response(prompt)
    return {"jewelry_recommendations": result}

# Simple page route for the jewelry planner
@app.get("/jewelry-planner")
async def jewelry_planner():
    return {"page": "Jewelry Budget Planner"}
```

- **/generate-jewelry: accepts budget, style and occasion to return suitable jewelry options.**

- **get home recommendations by using the function**

```
def get_home_recommendations(total_budget: float, needs: list[str]):
    catalog = {
        "kitchen": ("Mixer Grinder", 3500),
        "lighting": ("Smart LED Light", 1200),
        "storage": ("Storage Organizer", 900),
        "cleaning": ("Robot Vacuum", 18000),
    }

    recommendations = []
    for need in needs:
        if need.lower() in catalog:
            name, price = catalog[need.lower()]
            if price <= total_budget:
                recommendations.append({
                    "product": name,
                    "price": price,
                    "need": need
                })
    return recommendations

@router.post("/generate-home")
async def generate_home(data: HomeBudgetInput):
    items = get_home_recommendations(data.total_budget, data.needs)
    return {"home_recommendations": items}
```

← PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...

```
class JewelryInput(BaseModel):
    total_budget: float
    style: str = "any"
    occasion: str = "any"

JEWELRY_CATALOG = [
    {"name": "Silver Stud Earrings", "material": "Silver",
     "price": 799, "style": "minimal", "occasion": "daily"},
    {"name": "Pearl Necklace", "material": "Pearl",
     "price": 1499, "style": "classic", "occasion": "party"},
    {"name": "Gold-Plated Bracelet", "material": "Gold Plated",
     "price": 1999, "style": "traditional", "occasion": "wedding"},
    {"name": "Oxidised Jhumka", "material": "Alloy",
     "price": 699, "style": "traditional", "occasion": "daily"}
]

@router.post("/jewelry-recommendations")
async def jewelry_recommendations(data: JewelryInput):
    matches = []

    for item in JEWELRY_CATALOG:
        style_ok = data.style == "any" or item["style"] == data.style
        occasion_ok = (
            data.occasion == "any"
            or item["occasion"] == data.occasion
        )

        if item["price"] <= data.total_budget and style_ok and occasion_ok:
            matches.append(item)

    matches.sort(key=lambda item: item["price"])

    return {
        "budget": data.total_budget,
        "recommendations": matches
    }
```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

```
def create_jewelry_links(items):
    stores = {
        "Amazon": "https://www.amazon.in/s?k=",
        "Flipkart": "https://www.flipkart.com/search?q="
    }

    for item in items:
        query = item["name"].replace(" ", "+")
        item["links"] = {
            store: base_url + query
            for store, base_url in stores.items()
        }

    return items


def filter_jewelry(items, budget):
    return [
        item for item in items
        if item["price"] <= budget
    ]
```

Activity 2.3: Login and Register page routings

- Login page

`/login`: Authenticates user credentials and initiates a session upon successful login.

```
class RegisterInput(BaseModel):
    username: str
    email: str
    password: str


class LoginInput(BaseModel):
    email: str
    password: str


def create_password_hash(password: str) -> str:
    return pwd_context.hash(password)


@app.post("/register")
async def register_user(data: RegisterInput):
    existing = find_user_by_email(data.email)

    if existing:
        raise HTTPException(
            status_code=400,
```

- Register Page

`/register`: Handles new user registration by accepting and securely storing user credentials.

```
@app.post("/login")
async def login_user(data: LoginInput):
    user = find_user_by_email(data.email)

    if not user or not pwd_context.verify(
        data.password, user["password"]
    ):
        raise HTTPException(
            status_code=401,
            detail="Invalid email or password"
        )

    token = create_access_token(
        {"sub": user["email"]}
    )

    return {
        "message": "Login successful",
        "access_token": token,
        "token_type": "bearer"
    }
```

- Logout Page

`/logout`: Terminates the current user session and redirects to the login page.

```
from fastapi import APIRouter, Depends, HTTPException
from datetime import datetime, timedelta

router = APIRouter()

@router.post("/session/logout")
async def logout_user(current_user=Depends(get_current_active_user)):
    """End the active PocketSmart AI session."""
    username = current_user.username
    if username in active_sessions:
        active_sessions.pop(username)

    return {
        "message": "Logout successful",
        "username": username,
        "logged_out_at": datetime.utcnow().isoformat()
    }
```

Activity 2.4: Ensure modular code structure

- **`/token`**: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```
@router.post("/session/create")
async def create_session(data: SessionInput):
    # Create a session for an authenticated user
    if not verify_user(data.email, data.password):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token = create_access_token({"sub": data.email})
    expires_at = datetime.utcnow() + timedelta(hours=2)
    active_sessions[data.email] = {
        "token": token,
        "created_at": datetime.utcnow().isoformat(),
        "expires_at": expires_at.isoformat()
    }

    return {
        "message": "Session created",
        "access_token": token,
        "expires_at": expires_at.isoformat()
    }
```

• **`/session-info`**: Retrieves metadata about the current user session, such as user ID and login status.
• **`/session-data`**: Returns detailed session-specific data used for personalization and recommendation tracking.

```python
@router.get("/session-info")
async def session_info(current_user=Depends(get_current_active_user)):
    """Return active session details for PocketSmart AI."""
    session = active_sessions.get(current_user.username)

    if not session:
        raise HTTPException(status_code=404, detail="No active session")

    return {
        "username": current_user.username,
        "created_at": session["created_at"],
        "expires_at": session["expires_at"],
        "status": "active"
    }

@router.put("/session/preferences")
async def update_preferences(data: PreferenceInput,
                             current_user=Depends(get_current_active_user)):
    # Save budget preferences for personalized recommendations
    user_preferences[current_user.username] = {
        "budget": data.budget,
        "category": data.category,
        "store": data.preferred_store
    }
```

PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...

This milestone focuses on implementing the FastAPI-powered backend for **PocketSmart AI**. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in gemini_utils.py. The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains.It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

- **Activity 3.1: Define FastAPI Routes**
  - Home Planner (`/generate-home`): Accepts home-related preferences and budget to return tailored product suggestions.

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel

router = APIRouter()

class HomeBudgetInput(BaseModel):
    total_budget: float
    needs: list[str] = []
    preferred_store: str = "any"

@router.post("/generate-home")
async def generate_home(data: HomeBudgetInput,
                        current_user=Depends(get_current_active_user)):
    prompt = (
        f"Create home-product recommendations under Rs.{data.total_budget}. "
        f"Needs: {', '.join(data.needs)}. Store: {data.preferred_store}. "
        "Return product, price, store and URL."
    )
    recommendations = await generate_ai_response(prompt)
    save_to_history(current_user.username, "home", data.model_dump(), recommendations)
    return {"recommendations": recommendations}

@router.get("/home-history")
async def home_history(current_user=Depends(get_current_active_user)):
    return get_user_history(current_user.username, "home")
```

  - Party Planner (`/generate-party`): Processes party details and guest count to recommend venue, food, and decoration items.

```
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

class PartyBudgetInput(BaseModel):
    total_budget: float
    party_type: str
    guests: int
    food: str = "any"
    venue_type: str = "any"

@router.post("/generate-party")
async def generate_party(data: PartyBudgetInput):
    prompt = (
        f"Plan a {data.party_type} party for {data.guests} guests "
        f"within Rs.{data.total_budget}. Venue: {data.venue_type}. "
        f"Food preference: {data.food}. Return venue, catering "
        "and decoration options with estimated prices."
    )
    result = await generate_ai_response(prompt)
    return {"party_recommendations": result}

@router.get("/party-history/{username}")
async def party_history(username: str):
    return get_user_history(username, "party")
```

- Jewelry Planner (`/generate-jewelry`): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...

PocketSmart AI – Budget Recommendation Coding

```
# PocketSmart AI - Budget Recommendation
def recommend_by_budget(items, budget, category="any"):
    results = []

    for item in items:
        category_ok = (
            category == "any"
            or item.get("category") == category
        )

        if item["price"] <= budget and category_ok:
            results.append({
                "name": item["name"],
                "price": item["price"],
                "category": item.get("category", "general"),
                "store": item.get("store", "")
            })

    results.sort(key=lambda x: x["price"])

    return {
        "budget": budget,
        "category": category,
        "recommendations": results
    }

# Generate AI-friendly recommendation summary
def create_summary(recommendations):
    return [
        f'{item["name"]} - Rs.{item["price"]}'
        for item in recommendations
    ]
```

Recommendation History Coding

```
# Recommendation history endpoint
@router.get("/history")
async def recommendation_history(
    current_user=Depends(get_current_active_user)
):
    history = get_user_history(current_user.username)

    history_data = []
    for item in sorted(
        history, key=lambda x: x.timestamp, reverse=True
    ):
        history_data.append({
            "id": item.id,
            "timestamp": item.timestamp,
            "type": item.recommendation_type,
            "summary": item.result_summary
        })

    return {"history": history_data}
```

- User Auth (`/register`, `/login`, `/logout`, `/token`): Manages user registration, authentication, and session control.
- Session Info (`/session-info`, `/session-data`): Retrieves user-specific session data for personalized AI interaction.

- **Activity 3.2: Modular Architecture Setup**

`/recommendations-details`: Returns detailed AI-generated product recommendations based on user budget,

Recommendation Details Coding

```
# Get complete details for one recommendation
@router.get("/recommendation-details/{recommendation_id}")
async def recommendation_details(
    recommendation_id: str,
    current_user=Depends(get_current_active_user)
):
    records = get_user_history(current_user.username)

    for item in records:
        if str(item.id) == str(recommendation_id):
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.result_summary
            }

    raise HTTPException(
        status_code=404,
        detail="Recommendation not found"
    )
```

- **Activity 3.3: CORS & Static Routing**

```
# Step 3: Build recommendation statistics
@router.get("/recommendation-stats")
async def recommendation_stats(
    current_user=Depends(get_current_active_user)
):
    records = get_user_history(current_user.username)
    return {"total_recommendations": len(records)}
```

- **Activity 3.4: Startup and main function**

`/startup`: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```
@app.on_event("startup")
async def startup_services():
    # Initialize PocketSmart application state
    app.state.service_status = "ready"
    app.state.active_requests = 0

    print("PocketSmart AI services initialized")

    # Prepare recommendation cache
    app.state.recommendation_cache = {}

@app.middleware("http")
async def track_requests(request, call_next):
    app.state.active_requests += 1
    try:
        response = await call_next(request)
        return response
    finally:
        app.state.active_requests -= 1

# The startup function prepares services before requests are accepted.
```

`__main__`: Entry point of the FastAPI application that starts the server using `uvicorn` when the script is run directly.

```
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart AI...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

MILESTONE 4: UI Development

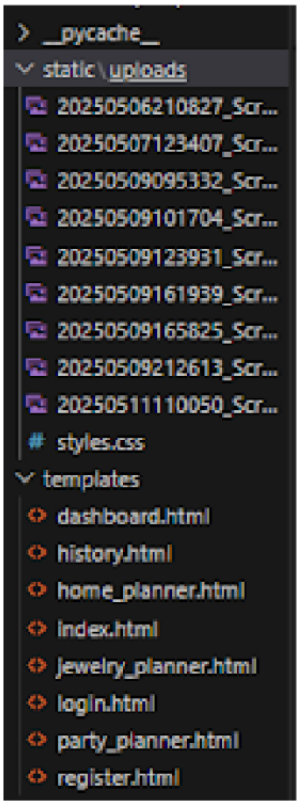
This milestone focuses on developing a **lightweight, responsive, and intuitive frontend** for **PocketSmart AI** using **HTML**, **CSS**, and **Jinja2 templates**. The interface enables users to interact seamlessly with the system's

structured AI recommendations, and includes authentication and session-based personalization.

Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
- Implement individual forms for each planner module connected to FastAPI POST routes
- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:



MILESTONE 5: Testing & Optimization

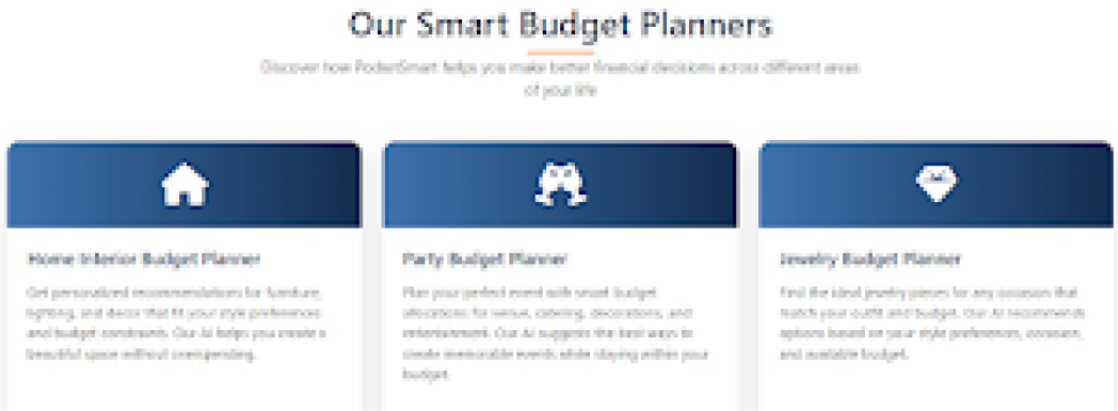
This milestone focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories—Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across devices.

- Home decor setups from platforms like IKEA, Amazon
- Party packages including food (Swiggy/Zomato), venues (OYO), and decor
- Jewelry suggestions matched with outfit styles, occasions, and image inputs

• **Home Page**

Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user actions.

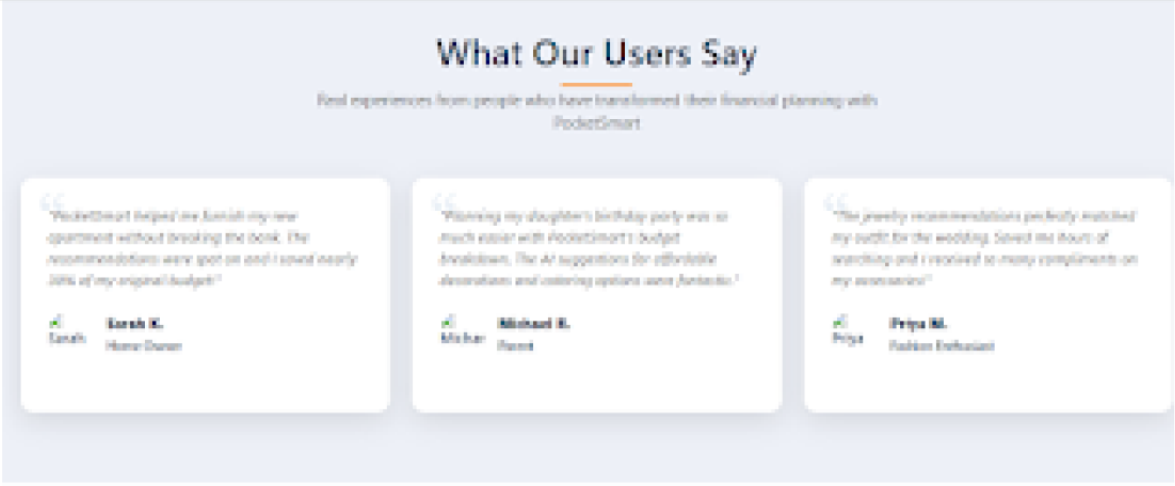


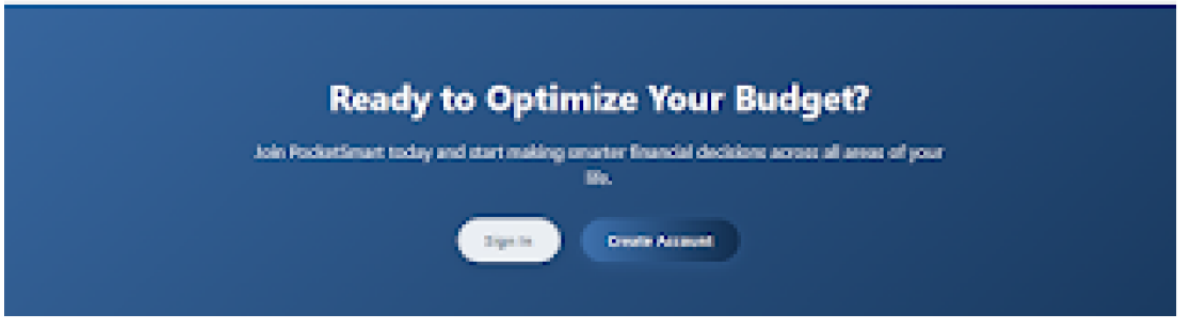


• **Testimonials Page**

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform's effectiveness.

← PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...



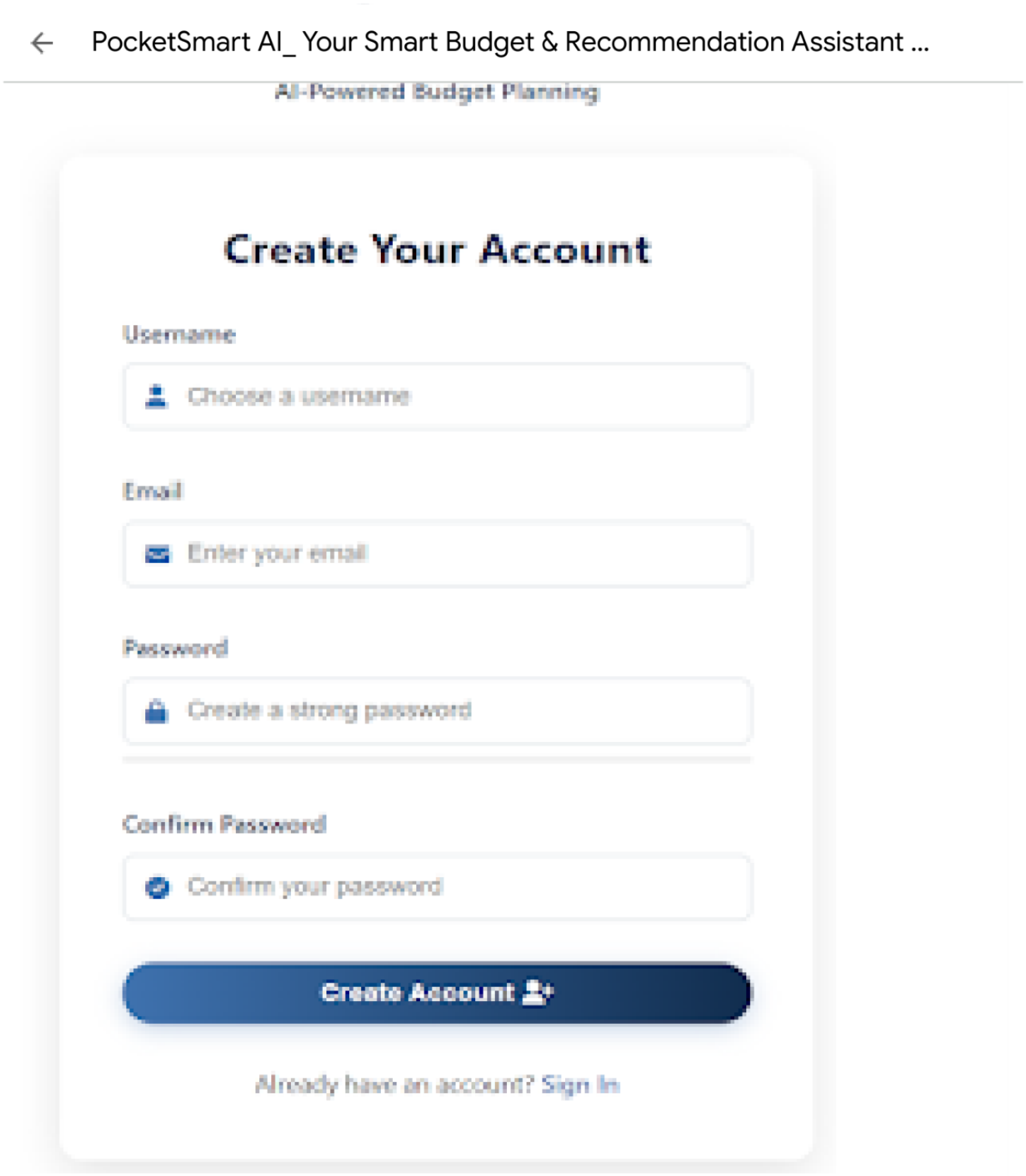


- **Footer** Part

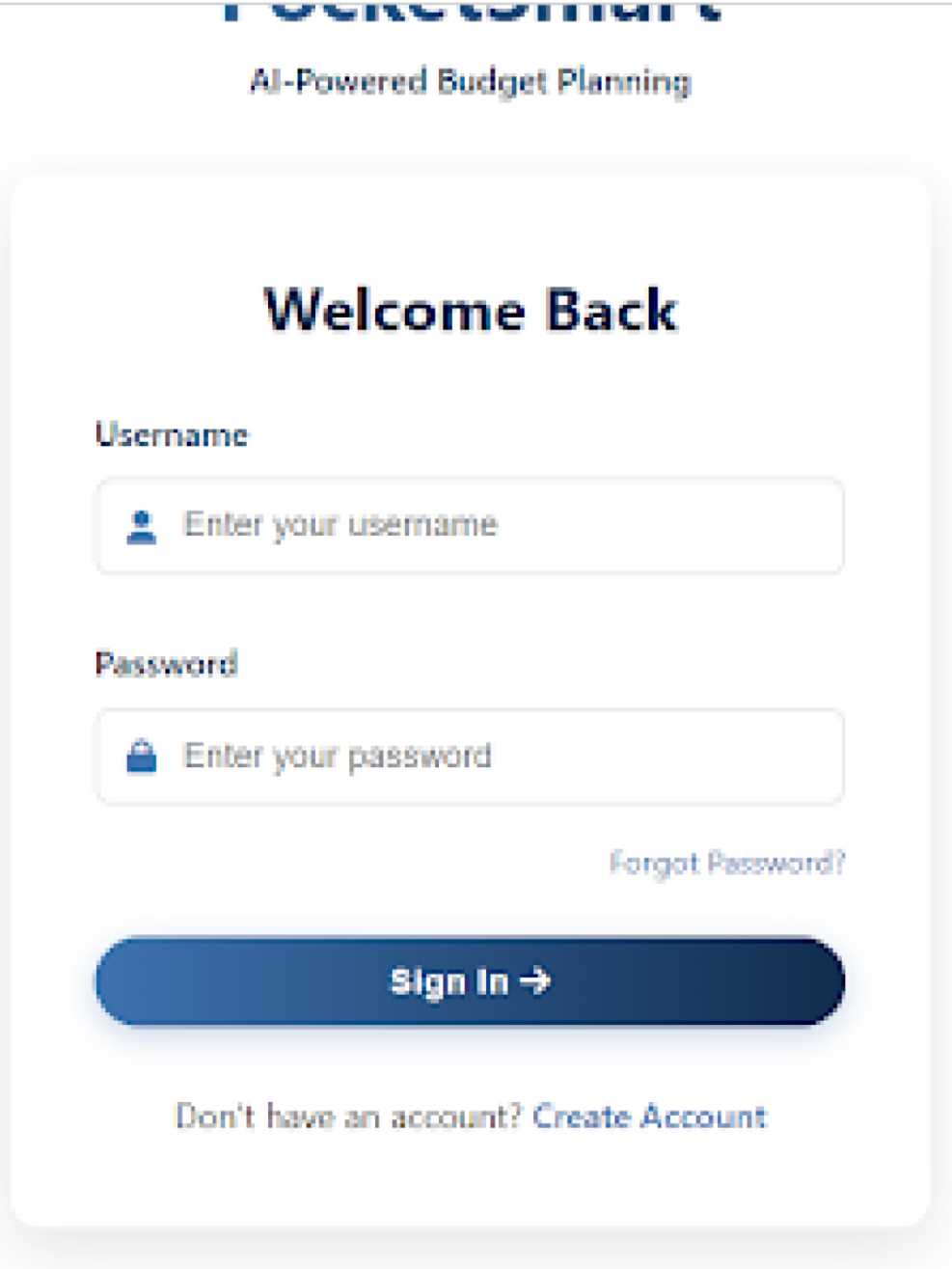


- **Register Page**

Register Page: Allows new users to create an account to access personalized budget recommendations.

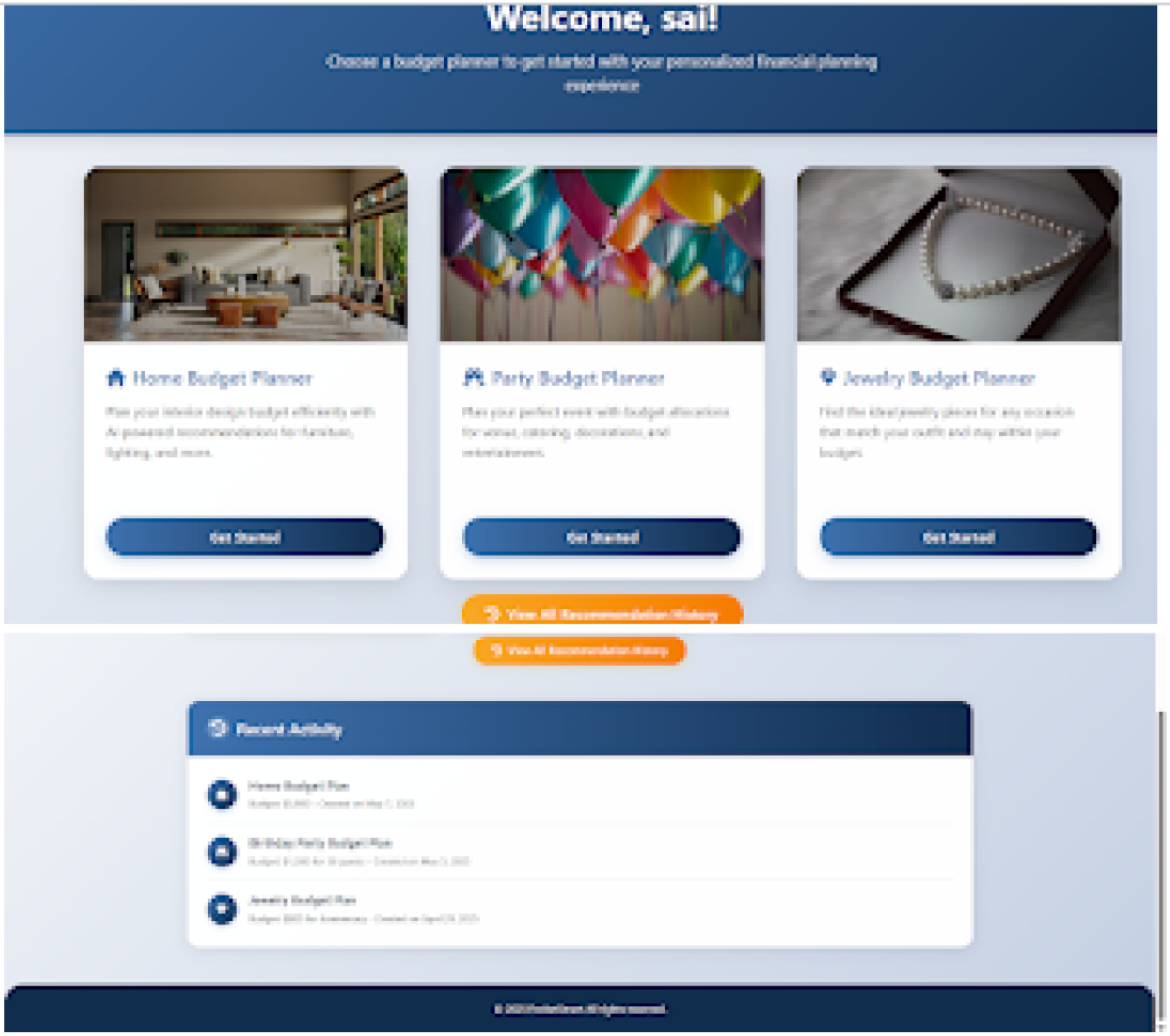


• **Login Page**[a]
Login Page: Authenticates existing users to access their dashboard and saved data.

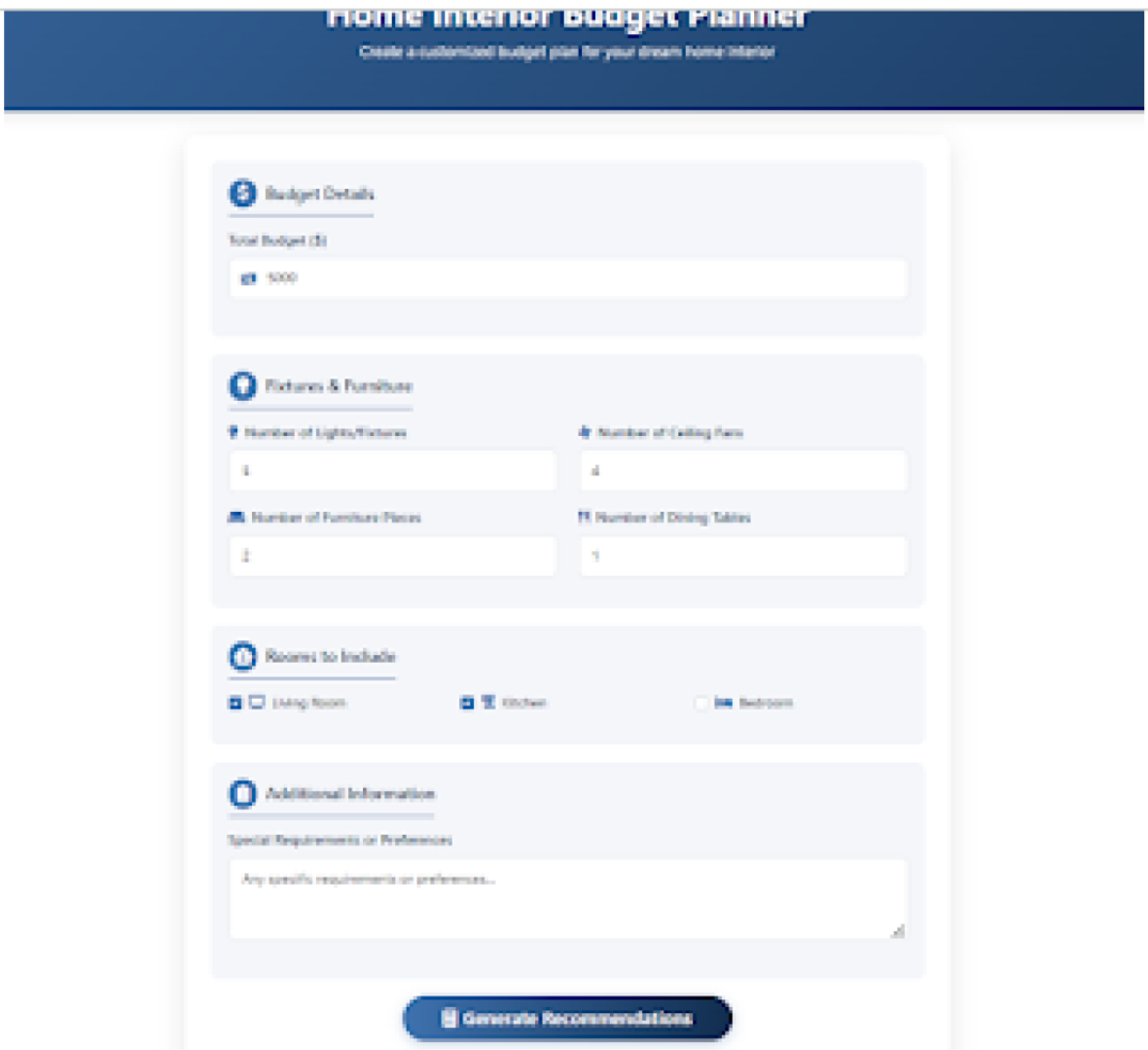


- **User Dashboard**

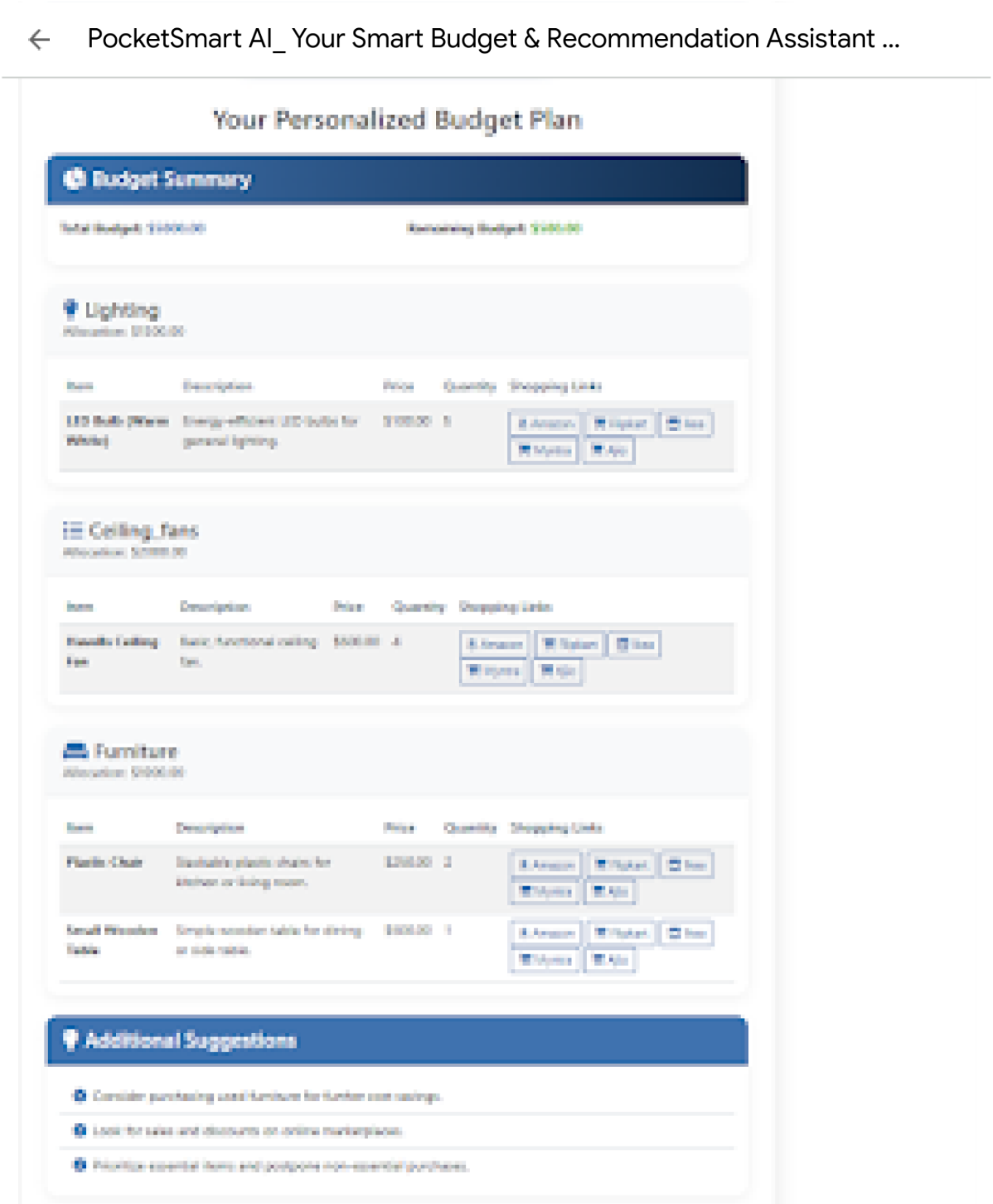
User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.



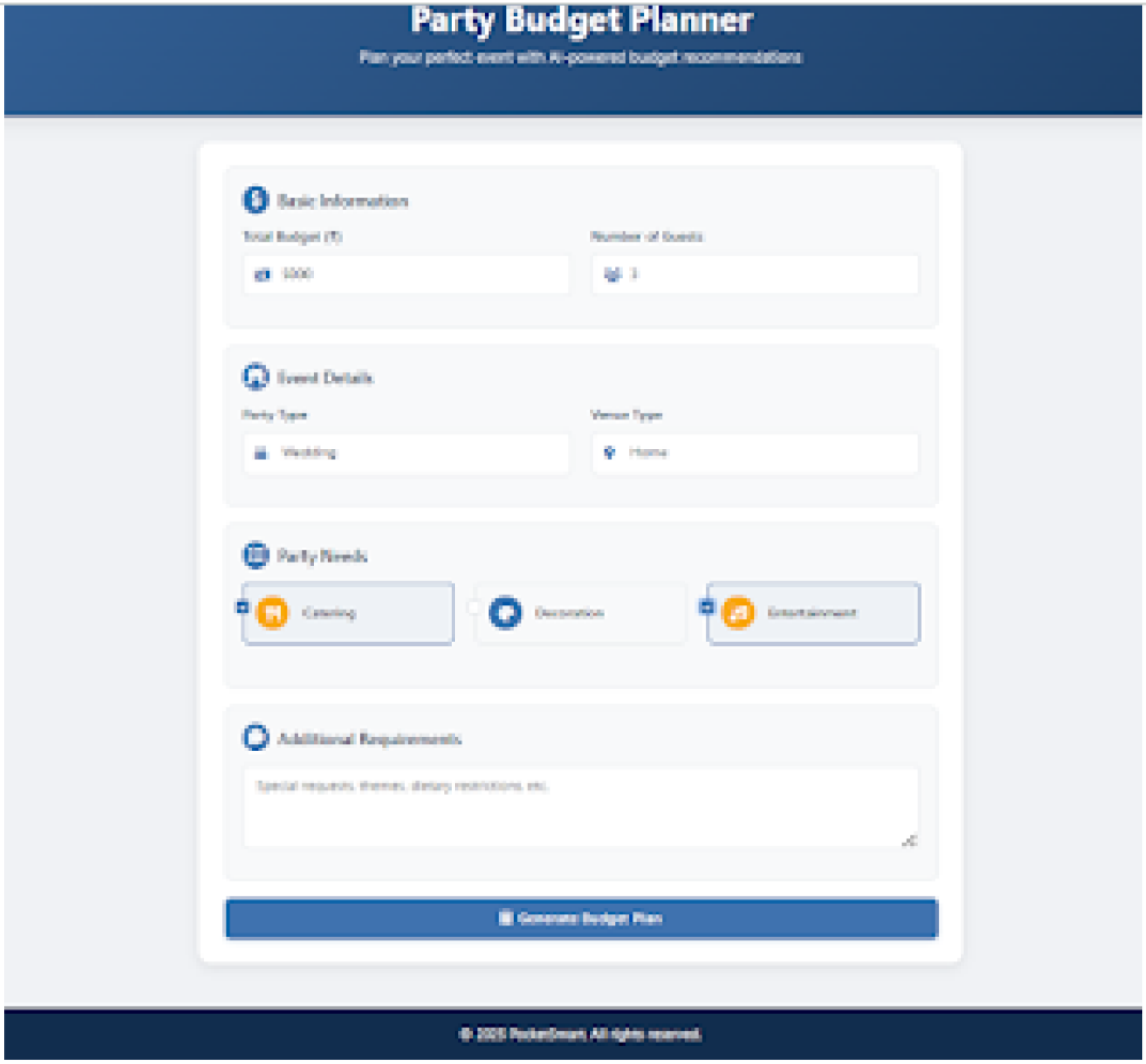
Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.



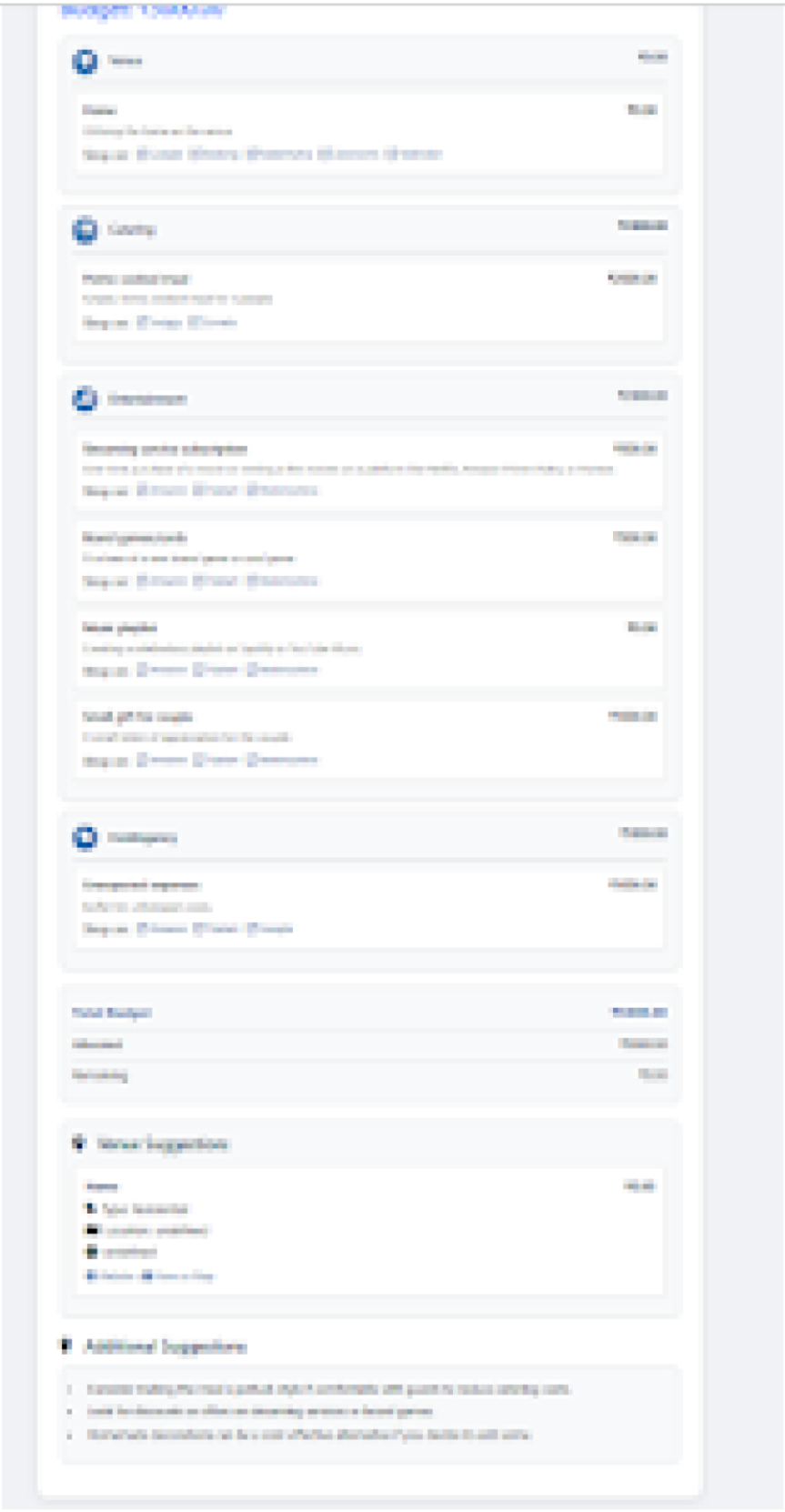
Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.



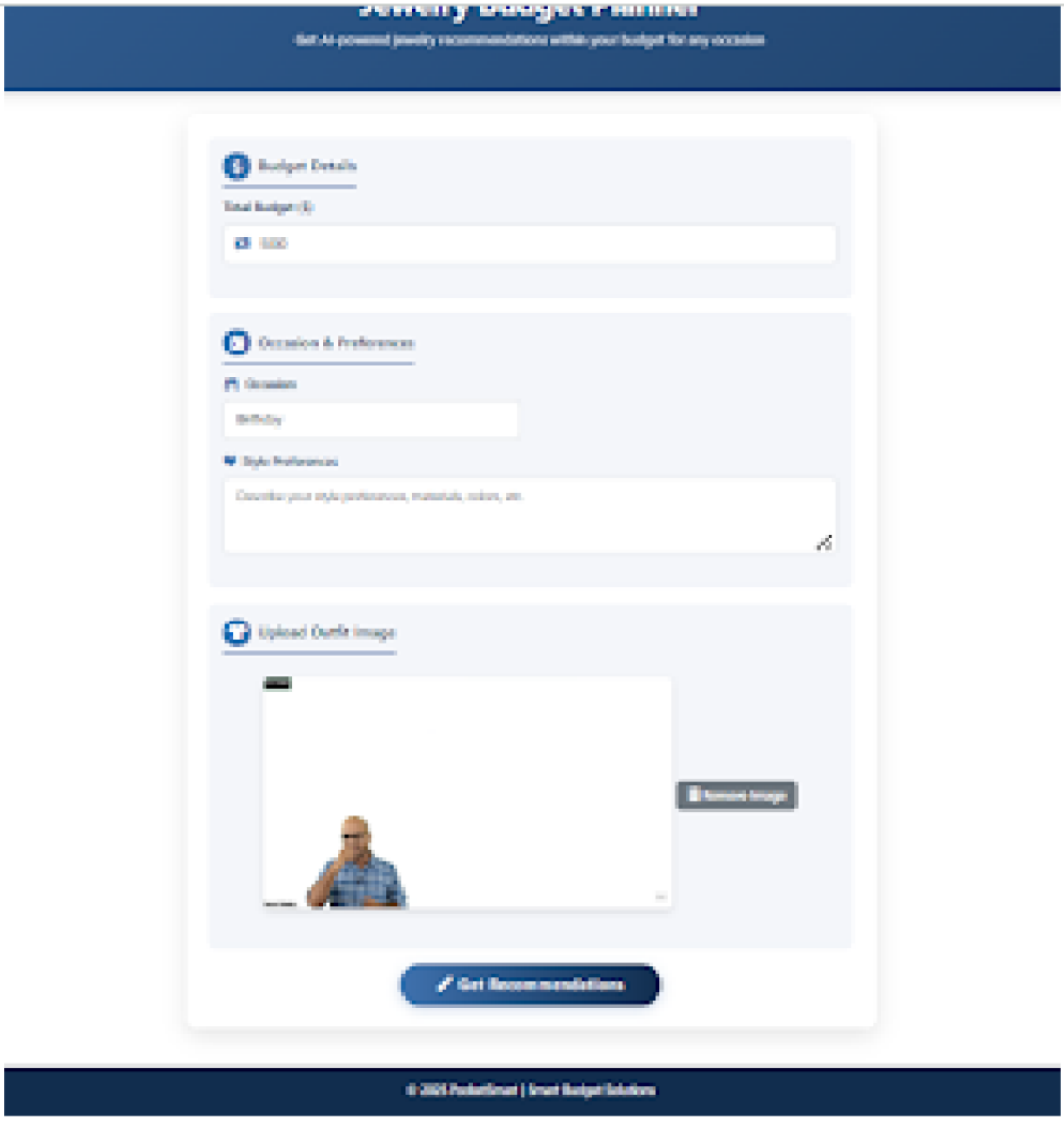
Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count



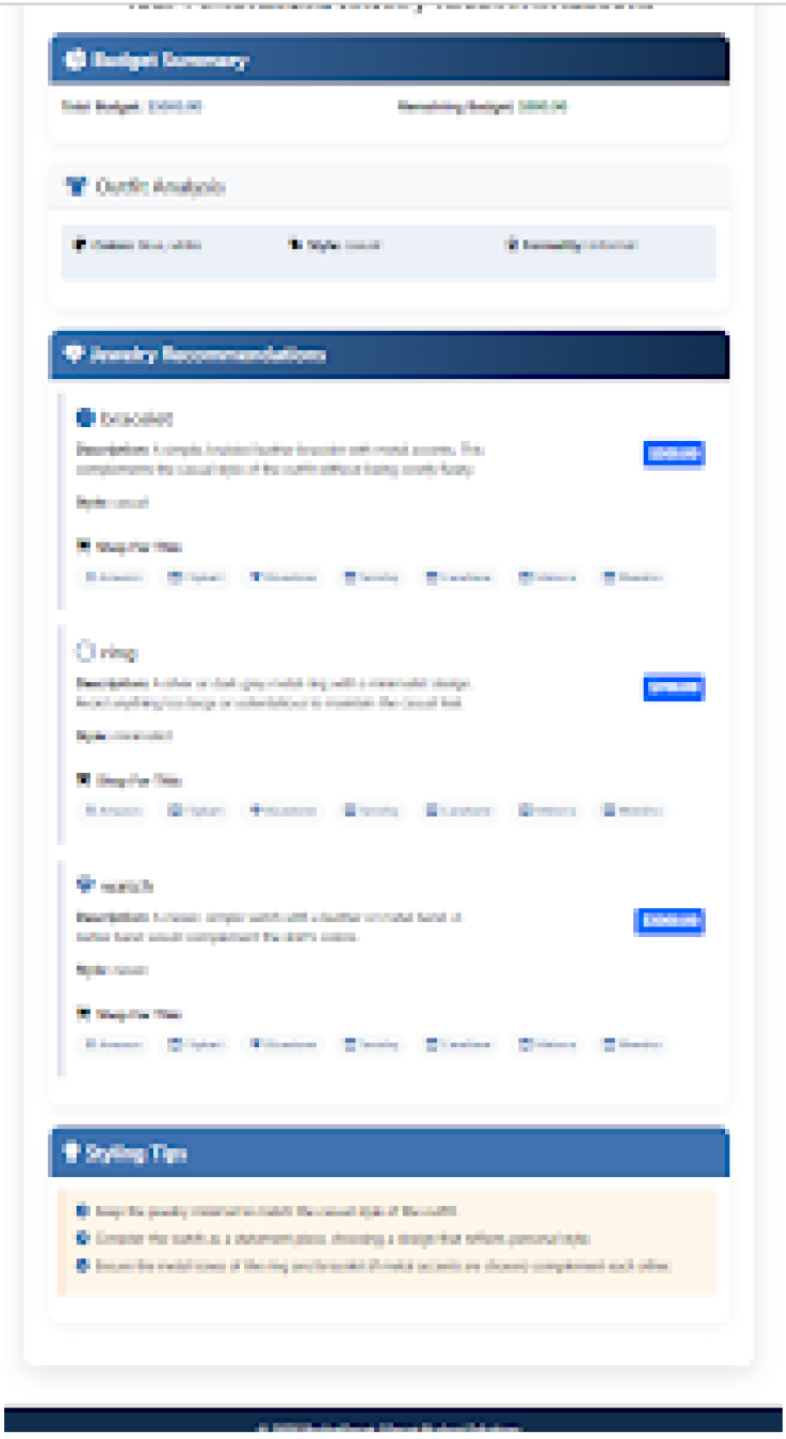
Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.



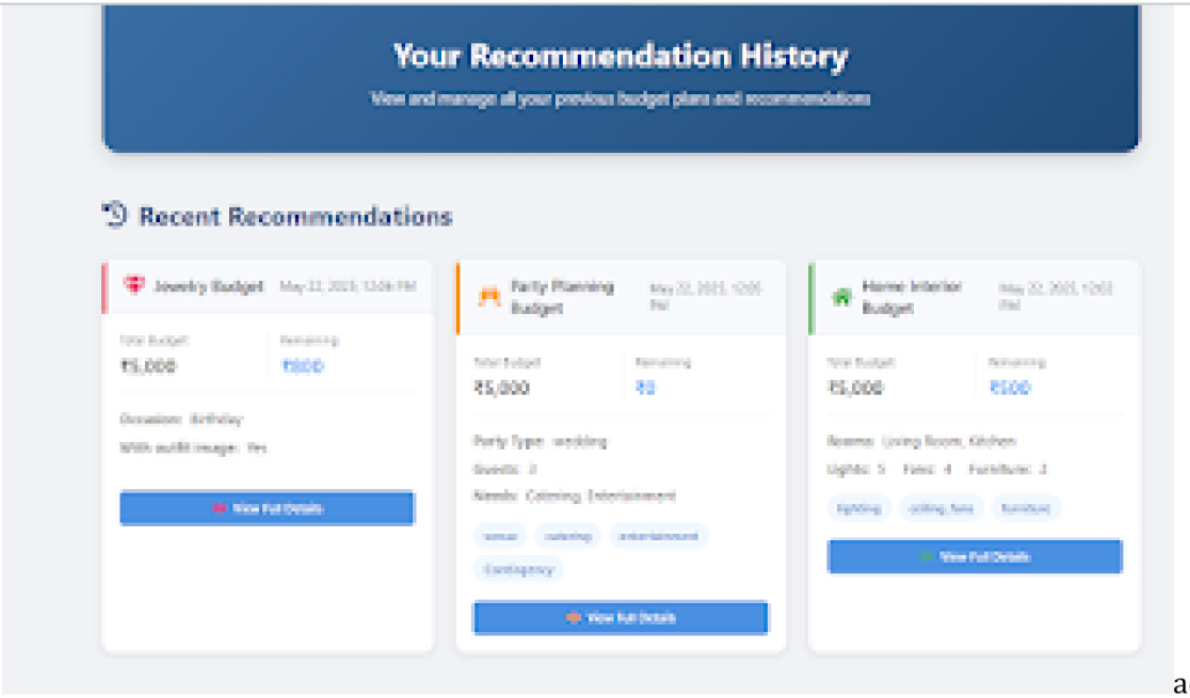
Jewelry Budget Planner Page: Provides jewelry recommendations aligned with occasion and outfit style, using text and optional image inputs.



Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.



Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.


Your Recommendation History
View and manage all your previous budget plans and recommendations
Recent Recommendations
Jewelry Budget
Total Budget
₹5,000
Remaining
₹600
Occasion: Birthday
With outfit image: Yes
View Full Details
Party Planning Budget
Total Budget
₹5,000
Remaining
₹0
Party Type: wedding
Guests: 2
Needs: Catering, Entertainment
venue
catering
entertainment
Contingency
View full Details
Home Interior Budget
Total Budget
₹5,000
Remaining
₹100
Rooms: Living Room, Kitchen
Lights: 5
Fans: 4
Furniture: 2
lighting
ceiling fans
furniture
View Full Details

aq

Conclusion

PocketSmart AI is an AI-powered budget and recommendation assistant developed to simplify planning for everyday lifestyle requirements. The system combines user budgets, preferences, and contextual inputs to provide personalized suggestions for home interiors, party planning, and jewelry selection.

The application uses FastAPI as the backend framework and integrates Gemini 1.5 Flash Pro to process user information and generate context-aware recommendations. It can also work with optional image inputs, particularly for jewelry suggestions based on outfit and occasion.

By connecting recommendation workflows with platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO, PocketSmart AI helps users explore suitable products and services while considering their defined budget.

The project demonstrates how generative AI, web technologies, and personalized planning can be combined in a single user-friendly system. Its modular backend, planner-based structure, authentication features, and responsive interface provide a practical foundation for future improvements and additional recommendation categories.

Overall, PocketSmart AI aims to make budgeting and lifestyle planning more convenient, personalized, and efficient by bringing intelligent recommendations into one accessible platform.